

A B C A A B C A A A $k=2$

valid window

make all chars
the most frequent one

window length - max freq $\leq k \Leftrightarrow$ valid window

A B C A A B C A A A

yes = X X X A X X
7

- if valid window - expand one step to the right
- if not - slide the window to the right

- Counts of each char in the window at most 26 entries
 - For a window, go through counts map/list & figure out max freq
 - $O(26n)$

- $\{A:3, B:4\}$
 - Maintain max freq in $\Theta(1)$ time by maintaining:
 - char to counts map
 - counts to char map

A A A A B C C C C C C C $\text{res} = 7$

$k=1$

A: 0, B: 1, C: 6

max-freq = 6

MaxPQ

insert(val)

removeMax()

getMax()

size()

isEmpty()

IndexMaxPQ

insert(index, val)

change(index, val)

contains(index) $\rightarrow \Theta(1)$

remove(index)

getMax() $\rightarrow \Theta(1)$

getMaxIndex() $\rightarrow \Theta(1)$

removeMax()

size() $\rightarrow \Theta(1)$

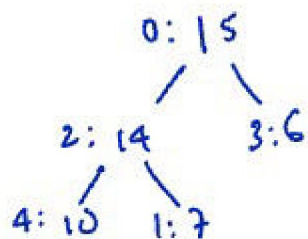
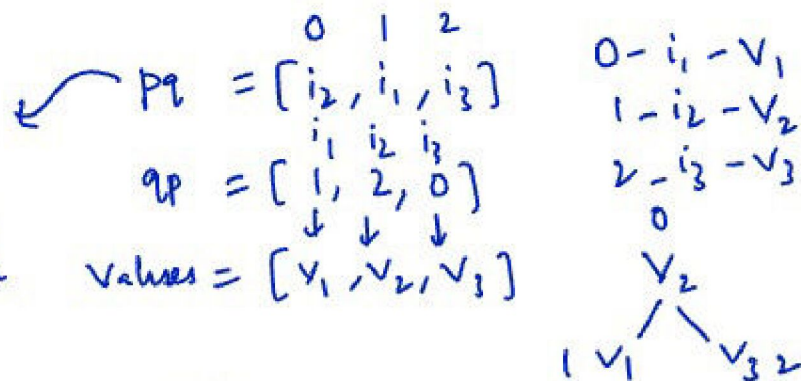
isEmpty() $\rightarrow \Theta(1)$

0	1	2	3	4	5	6	7
		2	3%3	4%3			
			=0	=1			

index in PQ = $ind \% k$

remainders
when divided k : $[0, k-1]$
 k remainders possible

array
forming
a complete
binary tree



	0	①	2	3	4	
PQ	2	0	3	4	1	
qP	1	4	0	2	3	$\rightarrow n$ elements 0 to $n-1$
values	15	7	14	6	10	

Monotonic queue

0	1	2	3	4	5	6	7	8	9	
4	2	100	100	20	19	13	14	13	13	25
100	100	100	100	20	19	14	14	13	13	

$(100, 3), (20, 4), (19, 5), (14, 7), (13, 9)$

	0	1	2	3	4	5	6	7
		3	-1	-3	5	3	6	7
prefix max	[1	3	3]	[-3	5	5]	[6	7]
suffix max	[3	3	-1]	[5	5	3]	[7	7]

$$\max(i, j) = \max(\text{suffix}[i], \text{prefix}[j])$$

[A D O B E C O D E B A] N C A

{A: 2
B: 2
C: 1}

min = 11

A B C A uc = 3

{A: 2
B: 1
C: 1}

Count = 3

↓
#char for which
count is met
(≤ 0 in the map)